A COMPLETE BEGINNER'S COURSE · LINUX EDITION

Apache Maven, from zero to multi-module

You know Java. You have never used a build tool. By the end of this guide you will understand what Maven does, why it does it that way, and every command you need day to day — while building one small project, *bookshop*, chapter by chapter.

CONTENTS

CHAPTER 1

What Maven is & installing it

Compiling one Java file is easy: `javac Main.java`. But a real project has hundreds of source files, test files that must *not* ship, third-party libraries with their own libraries, and a final `.jar` to produce. Maven is a **build tool**: it compiles, tests, packages, and manages libraries for you, all from one command.

Its core idea is **convention over configuration**. Instead of telling Maven where your sources live and what to do with them, you put files in the standard places, describe your project in one file called `pom.xml`, and Maven already knows the rest.

Installing on Linux

Maven needs a JDK (11 or newer). The package manager route is the easiest:

```
# Debian / Ubuntu
sudo apt update && sudo apt install maven

# Fedora
sudo dnf install maven

# Arch
sudo pacman -S maven
```

For the very latest version, download the binary tarball from maven.apache.org instead, unpack it to `/opt/maven`, and add its `bin/` to your PATH:

```
wget https://dlcdn.apache.org/maven/maven-3/3.9.9/binaries/apache-maven-3.9.9-
bin.tar.gz
sudo tar -xzf apache-maven-3.9.9-bin.tar.gz -C /opt
echo 'export PATH=/opt/apache-maven-3.9.9/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
```

Verify with `mvn -v`:

```
$ mvn -v
Apache Maven 3.9.9
Maven home: /opt/apache-maven-3.9.9
Java version: 21.0.4, vendor: Eclipse Adoptium
Default locale: en_US, platform encoding: UTF-8
OS name: "linux", arch: "amd64"
```

COMMANDS IN THIS CHAPTER

```
mvn -v  — show Maven, Java and OS versions
mvn -h  — list all command-line options
```

CHAPTER 2

Your first project

Maven can generate a project skeleton from a template called an **archetype**. We'll create the project we will grow for the rest of this guide: *bookshop*, a tiny app that manages a list of books.

```
mvn archetype:generate \
  -DgroupId=com.example \
  -DartifactId=bookshop \
  -DarchetypeArtifactId=maven-archetype-quickstart \
  -DarchetypeVersion=1.5 \
  -DinteractiveMode=false
```

This creates the **standard directory layout** — the convention Maven relies on:

```
bookshop/
├── pom.xml                          ← the project descriptor
└── src/
    ├── main/
    │   ├── java/com/example/App.java     ← application code
    │   └── resources/                    ← config files, shipped in the jar
    └── test/
        ├── java/com/example/AppTest.java ← test code (never shipped)
        └── resources/                    ← test-only config
```

Now compile and run it. Everything Maven produces goes into a `target/` folder it creates next to `pom.xml`:

```
$ cd bookshop
$ mvn compile
[INFO] --- compiler:3.13.0:compile (default-compile) @ bookshop ---
[INFO] Compiling 1 source file to /home/you/bookshop/target/classes
[INFO] BUILD SUCCESS

$ java -cp target/classes com.example.App
Hello World!
```

The first run downloads plugins and libraries into your **local repository** at `~/.m2/repository` — that's why it's slow once and fast forever after. And when you want a clean slate:

```
mvn clean        # deletes target/ entirely
```

COMMANDS IN THIS CHAPTER

```
mvn archetype:generate  — create a project from a template
mvn compile  — compile src/main/java into target/classes
mvn clean  — delete target/
```

CHAPTER 3

The POM, line by line

The **P**roject **O**bject **M**odel is Maven's single source of truth. Here is our bookshop `pom.xml`, trimmed to the parts that matter:

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>      <!-- who: your organization -->
  <artifactId>bookshop</artifactId>   <!-- what: this project's name -->
  <version>1.0-SNAPSHOT</version>     <!-- which: current version -->
  <packaging>jar</packaging>          <!-- output type (default: jar) -->

  <properties>
    <maven.compiler.release>21</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
</project>
```

The trio `groupId : artifactId : version` — the **GAV coordinates** — uniquely identifies every artifact in the Maven universe, yours and every library you'll ever download. `-SNAPSHOT` means "still in development, may change".

Properties are variables. `maven.compiler.release` sets the Java version; define your own (say `<junit.version>5.11.0</junit.version>`) and reference it anywhere as `${junit.version}`.

Your POM silently inherits hundreds of defaults from Maven's built-in *Super POM*. To see the fully-resolved result — every default made explicit — run:

```
mvn help:effective-pom
```

COMMANDS IN THIS CHAPTER

```
mvn help:effective-pom  — show the POM with all inherited defaults
mvn help:describe -Dplugin=compiler  — explain any plugin
```

CHAPTER 4

The build lifecycle

This is the single most important concept in Maven. A build is a fixed sequence of **phases**. When you run a phase, Maven runs *every phase before it*, in order. That's the whole trick.

PHASE	WHAT HAPPENS
`validate`	Check the POM is well-formed
`compile`	Compile `src/main/java` → `target/classes`
`test`	Compile and run `src/test/java`; fail the build on a failing test
`package`	Bundle classes into the artifact (a `.jar` for us)
`verify`	Run integration tests and quality checks
`install`	Copy the artifact into `~/.m2/repository` for other local projects
`deploy`	Upload the artifact to a shared remote repository

The default lifecycle, abridged — it has 23 phases in total, but these seven are the ones you type.

So `mvn package` means: validate, compile, test, *then* package. Two other small lifecycles exist alongside it: **clean** (delete `target/`) and **site** (generate a project report site). You can chain them:

```
mvn clean package      # wipe, then build the jar — the everyday command
mvn clean install      # wipe, build, and put the jar in ~/.m2
mvn verify             # everything up to integration tests
```

Try it on bookshop — you'll find `target/bookshop-1.0-SNAPSHOT.jar` waiting afterwards.

COMMANDS IN THIS CHAPTER

```
mvn validate · compile · test · package · verify · install · deploy
mvn clean package  — the everyday build
mvn site  — generate an HTML project report
```

CHAPTER 5

Dependencies

Bookshop should store books as JSON, so we need the Gson library. No downloading jars by hand: declare it by its GAV coordinates and Maven fetches it from **Maven Central** (the public repository) into `~/.m2`.

```
<dependencies>
  <dependency>
    <groupId>com.google.code.gson</groupId>
    <artifactId>gson</artifactId>
    <version>2.11.0</version>
  </dependency>
</dependencies>
```

Run `mvn compile` and Gson is on your classpath. Better: dependencies of your dependencies come along automatically — these are **transitive dependencies**. Inspect the whole tree:

```
$ mvn dependency:tree
[INFO] com.example:bookshop:jar:1.0-SNAPSHOT
[INFO] +- com.google.code.gson:gson:jar:2.11.0:compile
[INFO] |  \- com.google.errorprone:error_prone_annotations:jar:2.27.0:compile
[INFO] \- org.junit.jupiter:junit-jupiter:jar:5.11.0:test
```

Scopes — when a dependency is on the classpath

SCOPE	MEANING
`compile`	Default. Available everywhere, shipped with the app
`test`	Only for test code — e.g. JUnit. Never shipped
`provided`	Needed to compile, but the runtime supplies it (e.g. a servlet container)
`runtime`	Not needed to compile, needed to run (e.g. a JDBC driver)
`import`	Pull in a BOM's dependencyManagement (chapter 10)

If a transitive dependency causes trouble, cut it out with an **exclusion** inside the dependency that drags it in — and let `mvn dependency:analyze` tell you which declared dependencies you don't actually use.

COMMANDS IN THIS CHAPTER

```
mvn dependency:tree  — show all deps, incl. transitive
mvn dependency:analyze  — find unused / undeclared deps
mvn dependency:resolve  — download everything, list versions
mvn versions:display-dependency-updates  — what's outdated
```

CHAPTER 6

Plugins & goals

Here's the secret: **Maven itself does almost nothing**. Every real action — compiling, testing, jarring — is done by a **plugin**, and each plugin offers **goals** (individual tasks). The lifecycle phases from chapter 4 are just hooks that plugins bind their goals to: at the `compile` phase, the `compiler` plugin runs its `compile` goal.

You can also run a goal directly, without any lifecycle, using `plugin:goal` syntax — you've done it already with `archetype:generate` and `dependency:tree`.

Plugins are configured in the POM. Let's give bookshop a proper run command with the `exec` plugin:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.codehaus.mojo</groupId>
      <artifactId>exec-maven-plugin</artifactId>
      <version>3.3.0</version>
      <configuration>
        <mainClass>com.example.App</mainClass>
      </configuration>
    </plugin>
  </plugins>
</build>
```

```
$ mvn exec:java
Hello World!
```

Plugins you'll meet constantly

PLUGIN	JOB
`compiler`	Runs javac; Java version comes from `maven.compiler.release`
`surefire`	Runs unit tests during the `test` phase
`jar`	Builds the jar during `package`
`shade`	Builds a fat jar with all dependencies inside (chapter 8)
`exec`	Runs your main class from the build
`versions`	Reports and bumps outdated dependency versions

COMMANDS IN THIS CHAPTER

```
mvn <plugin>:<goal>  — run one goal directly
mvn exec:java  — run the configured main class
mvn help:describe -Dplugin=surefire -Ddetail  — list a plugin's goals
```

CHAPTER 7

Testing

Add JUnit 5 with `test` scope, write a test for bookshop's `Book` class, and the `test` phase does the rest.

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.11.0</version>
  <scope>test</scope>
</dependency>
```

```
// src/test/java/com/example/BookTest.java
package com.example;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class BookTest {
  @Test void titleIsRequired() {
    assertThrows(IllegalArgumentException.class, () -> new Book(""));
  }
}
```

```
$ mvn test
[INFO] --- surefire:3.2.5:test (default-test) @ bookshop ---
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Surefire finds test classes by name convention (`*Test`, `Test*`, `*Tests`). Day-to-day variations:

```
mvn test -Dtest=BookTest                 # one class
mvn test -Dtest=BookTest#titleIsRequired # one method
mvn package -DskipTests                  # compile tests but don't run them
mvn package -Dmaven.test.skip=true       # don't even compile them
```

A failing test fails the build — that's the point. Reports land in `target/surefire-reports/`.

COMMANDS IN THIS CHAPTER

```
mvn test  — run all unit tests
mvn test -Dtest=Name  — run one class / method
mvn package -DskipTests  — build without running tests
```

CHAPTER 8

Packaging & running

`mvn package` gives you `target/bookshop-1.0-SNAPSHOT.jar` — but `java -jar` refuses to run it: the jar has no main class in its manifest, and Gson isn't inside it. Two fixes.

1 — Declare the main class

```
<plugin>
  <artifactId>maven-jar-plugin</artifactId>
  <configuration>
    <archive><manifest>
      <mainClass>com.example.App</mainClass>
    </manifest></archive>
  </configuration>
</plugin>
```

2 — Bundle dependencies with the shade plugin

The **shade** plugin builds an *uber-jar* (fat jar): your classes plus every dependency, in one runnable file.

```
<plugin>
  <artifactId>maven-shade-plugin</artifactId>
  <version>3.6.0</version>
  <executions>
    <execution>
      <phase>package</phase>
      <goals><goal>shade</goal></goals>
    </execution>
  </executions>
</plugin>
```

Note the `<executions>` block — that's how you bind any plugin goal to a lifecycle phase yourself. Now:

```
$ mvn clean package
$ java -jar target/bookshop-1.0-SNAPSHOT.jar
Bookshop — 3 books loaded from books.json
```

COMMANDS IN THIS CHAPTER

```
mvn clean package  — build the (fat) jar
java -jar target/….jar  — run it
jar tf target/….jar  — peek inside the jar
```

CHAPTER 9

Profiles

A **profile** is a named bundle of POM overrides you can switch on per build — different settings for development vs. production, for example. Bookshop reads its database file from a property; profiles change which one:

```
<profiles>
  <profile>
    <id>dev</id>
    <activation><activeByDefault>true</activeByDefault></activation>
    <properties><books.file>books-sample.json</books.file></properties>
  </profile>
  <profile>
    <id>prod</id>
    <properties><books.file>/var/lib/bookshop/books.json</books.file>
</properties>
  </profile>
</profiles>
```

```
mvn package -P prod          # activate the prod profile
mvn package -P prod,ci       # several at once
mvn help:active-profiles     # which profiles are on right now?
```

Profiles can also activate themselves automatically — by JDK version, OS, an environment variable, or a file's presence — via richer `<activation>` rules. Use profiles sparingly: every profile doubles the number of builds you should test.

COMMANDS IN THIS CHAPTER

`mvn <phase> -P name` — build with a profile active

`mvn help:active-profiles` — list active profiles

`mvn help:all-profiles` — list every available profile

CHAPTER 10

Multi-module projects

Bookshop has grown: we want a `bookshop-core` library and a `bookshop-cli` app that uses it. Maven handles this with a **parent POM** that lists **modules**:

```
bookshop/
├── pom.xml              ← parent: packaging "pom", lists modules
├── bookshop-core/
│   └── pom.xml          ← library module
└── bookshop-cli/
    └── pom.xml          ← app module, depends on core
```

```
<!-- parent pom.xml -->
<groupId>com.example</groupId>
<artifactId>bookshop</artifactId>
<version>1.0-SNAPSHOT</version>
<packaging>pom</packaging>

<modules>
  <module>bookshop-core</module>
  <module>bookshop-cli</module>
</modules>

<dependencyManagement>         <!-- pins versions for all modules -->
  <dependencies>
    <dependency>
      <groupId>com.google.code.gson</groupId>
      <artifactId>gson</artifactId>
      <version>2.11.0</version>
    </dependency>
  </dependencies>
</dependencyManagement>
```

Each child declares `<parent>` and inherits its groupId, version, properties and plugin config. **`dependencyManagement`** pins versions in one place — children then declare dependencies *without* a version. The cli module depends on core like any other artifact:

```
<dependency>
  <groupId>com.example</groupId>
  <artifactId>bookshop-core</artifactId>
  <version>${project.version}</version>
</dependency>
```

Build from the parent and the **reactor** works out the right order (core before cli). Useful flags:

```
mvn install                       # build every module, in order
mvn -pl bookshop-cli package      # only this module…
mvn -pl bookshop-cli -am package  # …plus what it depends on
mvn -rf bookshop-cli install      # resume a failed build from here
mvn -T 1C install                 # parallel: one thread per CPU core
```

COMMANDS IN THIS CHAPTER

```
mvn -pl module  — build selected module(s)
mvn -am / -amd  — also build dependencies / dependents
mvn -rf module  — resume from a module
mvn -T 1C  — parallel build
```

CHAPTER 11

Repositories, install & deploy

Three places artifacts live, and Maven checks them in this order:

1. **Local repository** — `~/.m2/repository`, your on-disk cache. `mvn install` puts your own artifacts here.
2. **Maven Central** — the public repository nearly every open-source Java library is published to.
3. **Company repositories** — a private Nexus or Artifactory your team shares. `mvn deploy` uploads there, configured via `<distributionManagement>` in the POM and credentials in `~/.m2/settings.xml`.

`settings.xml` is per-user configuration that doesn't belong in the project: mirrors, proxies, credentials, default profiles. Useful network-related flags:

```
mvn -o package       # offline: use only ~/.m2, no downloads
mvn -U package       # force re-check remote SNAPSHOT updates
mvn dependency:purge-local-repository   # re-download this project's deps
```

When core's API stabilizes, drop the `-SNAPSHOT` and release `1.0`: release versions are immutable — a repository will refuse to overwrite one, which is exactly what makes builds reproducible.

COMMANDS IN THIS CHAPTER

`mvn install` — publish to your local ~/.m2

`mvn deploy` — publish to the shared remote repository

`mvn -o` — build offline · `mvn -U` — force snapshot updates

CHAPTER 12

Plugin directory

Chapter 6 explained *what* plugins are; this is the field guide to the twenty you'll actually meet, ordered by how often. Stars mark how soon you'll need each one. Remember the two ways any plugin runs: bound to a lifecycle phase (automatic), or invoked directly as `mvn plugin:goal`.

Core of every build ★★★★★

maven-compiler-plugin

How it works: binds to `compile` and `test-compile` and runs javac on main and test sources. **When to touch it:** to change the Java version or pass compiler flags — otherwise it just works.

```
<properties>
  <maven.compiler.release>21</maven.compiler.release>   <!-- the usual way -->
</properties>
<!-- or, for compiler args, configure the plugin itself: -->
<plugin>
  <artifactId>maven-compiler-plugin</artifactId>
  <configuration><compilerArgs><arg>-Xlint:all</arg></compilerArgs>
</configuration>
</plugin>
```

maven-surefire-plugin

How it works: binds to the `test` phase, finds classes named `*Test` / `Test*`, runs them, fails the build on failure. Reports land in `target/surefire-reports/`. **When to touch it:** to filter which tests run.

```
mvn test -Dtest=BookTest          # one class
mvn test -Dtest='*RepositoryTest' # by pattern
mvn package -DskipTests           # skip entirely
```

maven-jar-plugin

How it works: binds to `package` and zips `target/classes` into the jar. **When to touch it:** to set the manifest's main class so `java -jar` works (chapter 8's first fix).

```
<configuration>
  <archive><manifest><mainClass>com.example.App</mainClass></manifest></archive>
</configuration>
```

spring-boot-maven-plugin

How it works: adds a `repackage` goal at `package` that rewrites the plain jar into an executable Spring Boot jar (dependencies nested inside, special launcher in the manifest), plus a dev-time run goal. **When to use it:** every Spring Boot project — the Spring Initializr adds it for you.

```
mvn spring-boot:run        # run the app in place, with devtools reload
mvn package                # produces the executable boot jar
java -jar target/app.jar   # …which runs anywhere
```

In every serious project ★★★★☆

maven-resources-plugin

How it works: at `process-resources` it copies `src/main/resources` into `target/classes` so config files end up inside the jar. **When to touch it:** turn on *filtering* to substitute `${…}` placeholders in resource files with POM properties — pairs perfectly with chapter 9's profiles.

```
<build>
  <resources>
    <resource>
      <directory>src/main/resources</directory>
      <filtering>true</filtering>   <!-- app.properties:
books.file=${books.file} -->
    </resource>
  </resources>
</build>
```

maven-clean-plugin

How it works: it *is* `mvn clean` — deletes `target/`. **When to touch it:** only to make it delete extra generated folders.

```
<configuration>
  <filesets><fileset><directory>generated/</directory></fileset></filesets>
</configuration>
```

exec-maven-plugin

How it works: `exec:java` runs your `main()` inside Maven's JVM with the project classpath already set up; `exec:exec` forks any external program. **When to use it:** running a plain (non-Boot) app during development — chapter 6 set it up for bookshop.

```
mvn exec:java                                      # uses configured mainClass
mvn exec:java -Dexec.mainClass=com.example.App \
              -Dexec.args="add 'Clean Code'"       # override + pass arguments
```

maven-failsafe-plugin

How it works: surefire's sibling for *integration* tests. It finds `*IT` classes and runs them in the `integration-test` phase, but only fails the build later, at `verify` — so a test server started before the tests always gets shut down ("fail safe"). **When to use it:** tests that hit a real database, HTTP server, or container.

```
<plugin>
  <artifactId>maven-failsafe-plugin</artifactId>
  <version>3.2.5</version>
  <executions><execution>
    <goals><goal>integration-test</goal><goal>verify</goal></goals>
  </execution></executions>
</plugin>
```

maven-install-plugin & maven-deploy-plugin

How they work: they *are* the `install` and `deploy` phases — copy to `~/.m2`, upload to the remote repository (chapter 11). **When to touch them:** almost never, with one handy exception — registering a jar you only have as a file:

```
mvn install:install-file -Dfile=vendor-sdk.jar \
  -DgroupId=com.vendor -DartifactId=sdk -Dversion=1.0 -Dpackaging=jar
```

Packaging & quality ★★★☆☆

maven-shade-plugin

How it works: at `package` it unpacks every dependency and merges the classes into one fat jar; it can also *relocate* packages to dodge version clashes. **When to use it:** standalone CLI tools like bookshop (chapter 8 has the full setup).

maven-assembly-plugin

How it works: builds arbitrary distributions — ZIP, TAR, a folder layout — from an XML *descriptor*; the built-in `jar-with-dependencies` descriptor is a quick fat jar. **When to use it:** shipping an app with launch scripts, docs and config alongside the jar.

```
<configuration>
  <descriptorRefs><descriptorRef>jar-with-dependencies</descriptorRef>
</descriptorRefs>
</configuration>
mvn package assembly:single
```

jacoco-maven-plugin

How it works: attaches a Java agent before tests run to record which lines execute, then writes an HTML coverage report to `target/site/jacoco/`. **When to use it:** any project that tracks test coverage — usually wired into CI.

```
<plugin>
  <groupId>org.jacoco</groupId><artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.12</version>
  <executions>
    <execution><goals><goal>prepare-agent</goal></goals></execution>
    <execution><id>report</id><phase>test</phase>
      <goals><goal>report</goal></goals></execution>
  </executions>
</plugin>
```

spotbugs-maven-plugin

How it works: static analysis of the *compiled bytecode* for known bug patterns — null dereferences, bad equals/hashCode, resource leaks. **When to use it:** as a CI quality gate; `spotbugs:check` fails the build on findings, `spotbugs:gui` browses them.

maven-checkstyle-plugin

How it works: checks *source code style* — naming, imports, whitespace — against a rules file (e.g. `google_checks.xml`). **When to use it:** team projects where formatting arguments waste time; `checkstyle:check` fails the build on violations. SpotBugs finds *bugs*; Checkstyle polices *style*.

Occasional but worth knowing ★★☆☆☆

versions-maven-plugin

How it works: compares your declared versions against the repository and can rewrite the POM. **When to use it:** periodic dependency hygiene, and bumping the project version at release time.

```
mvn versions:display-dependency-updates   # what's outdated (read-only)
mvn versions:use-latest-releases          # rewrite POM to latest versions
mvn versions:set -DnewVersion=1.1.0       # bump the project's own version
```

maven-dependency-plugin

How it works: chapter 5's inspection tool — `dependency:tree`, `dependency:analyze`, plus `dependency:copy-dependencies` to dump every dependency jar into a folder. **When to use it:** debugging version conflicts, or building a classic `lib/` folder deployment.

maven-site-plugin

How it works: `mvn site` generates an HTML project site in `target/site/` — project info, dependency reports, and any reporting plugins (JaCoCo, Checkstyle) you've wired in. **When to use it:** teams that publish build reports internally.

maven-javadoc-plugin

How it works: runs the `javadoc` tool over your sources — `javadoc:javadoc` for browsable HTML in `target/site/apidocs/`, `javadoc:jar` for the `-javadoc.jar` Maven Central requires. **When to use it:** publishing a library.

maven-enforcer-plugin

How it works: runs *rules* at `validate` and stops the build immediately if one fails — before a wrong-JDK build wastes ten minutes. **When to use it:** any team project, to guarantee everyone builds with the same toolchain.

```
<plugin>
  <artifactId>maven-enforcer-plugin</artifactId>
  <executions><execution>
    <goals><goal>enforce</goal></goals>
    <configuration><rules>
      <requireJavaVersion><version>[21,)</version></requireJavaVersion>
      <requireMavenVersion><version>[3.9,)</version></requireMavenVersion>
      <dependencyConvergence/>   <!-- no conflicting transitive versions -->
    </rules></configuration>
  </execution></executions>
</plugin>
```

CHAPTER 13

Command cheat sheet

COMMAND	WHAT IT DOES
`mvn -v`	Show Maven & Java versions
`mvn archetype:generate`	Create a new project from a template
`mvn clean`	Delete target/
`mvn compile`	Compile main sources
`mvn test`	Run unit tests
`mvn package`	Build the jar/war into target/
`mvn verify`	Package + integration tests & checks
`mvn install`	Verify + copy artifact to ~/.m2
`mvn deploy`	Install + upload to remote repository
`mvn clean package`	The everyday full build
`mvn exec:java`	Run the configured main class
`mvn dependency:tree`	Show the full dependency tree
`mvn dependency:analyze`	Find unused / undeclared dependencies
`mvn help:effective-pom`	POM with all inherited defaults resolved
`mvn help:describe -Dplugin=X`	Explain a plugin and its goals
`mvn help:active-profiles`	List profiles active for this build
`mvn versions:display-dependency-updates`	List outdated dependencies
`mvn site`	Generate an HTML report site

Flags worth memorizing

FLAG	EFFECT
`-DskipTests`	Compile but don't run tests
`-Dtest=ClassName`	Run only matching tests
`-P name`	Activate a profile
`-pl module (-am / -amd)`	Build selected modules (+deps / +dependents)
`-rf module`	Resume a failed multi-module build
`-T 1C`	Parallel build, one thread per core

FLAG	EFFECT
`-o / -U`	Offline mode / force snapshot updates
`-q / -X`	Quiet output / full debug output
`-e`	Show full error stack traces
`-f path/pom.xml`	Build a POM other than ./pom.xml
`-N`	Non-recursive: parent only, skip modules

That's the whole toolbox. From here: explore `mvnw` (the Maven Wrapper, which pins a Maven version per project), and if you move to Spring Boot you'll find it is just Maven with a parent POM and one plugin — everything in this guide carries over.

CHAPTER 14

POM tag reference

Every tag a `pom.xml` can carry, grouped by what it does. Use this as a lookup when you meet an unfamiliar tag in someone else's POM.

1 · Project information

```
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>bookshop</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>
  <name>Bookshop</name>
  <description>A tiny book manager</description>
  <url>https://example.com/bookshop</url>
</project>
```

TAG	PURPOSE
`<modelVersion>`	POM model version — always 4.0.0
`<groupId>`	Organization / package identifier
`<artifactId>`	Project name
`<version>`	Project version (-SNAPSHOT = in development)
`<packaging>`	jar (default), war, pom, ear…
`<name> / <description> / <url>`	Human-readable metadata, shown in reports and repositories

2 · Properties

Reusable variables (chapter 3). Define once, reference anywhere as `${java.version}`.

```
<properties>
  <java.version>21</java.version>
  <junit.version>5.11.0</junit.version>
</properties>
```

3 · Dependencies — the full anatomy

Chapter 5 covered the everyday trio + scope; these are *all* the tags a dependency accepts:

```
<dependency>
  <groupId>…</groupId><artifactId>…</artifactId><version>…</version>
  <scope>test</scope>
  <optional>true</optional>
  <type>jar</type>
  <classifier>sources</classifier>
  <exclusions>
    <exclusion><groupId>…</groupId><artifactId>…</artifactId></exclusion>
  </exclusions>
</dependency>
```

TAG	PURPOSE
`<scope>`	compile / test / runtime / provided / import (chapter 5)
`<optional>`	True = projects depending on *you* don't inherit this dependency
`<type>`	jar (default), pom, war…
`<classifier>`	A variant artifact: sources, javadoc, an OS-specific build
`<exclusions>`	Cut unwanted transitive dependencies (chapter 5)

4 · dependencyManagement

Pins versions *without* adding the dependency — children/modules then declare it version-free (chapter 10). This is how Spring Boot's parent POM manages hundreds of library versions for you.

5 · build — plugins & friends

```
<build>
  <finalName>BookShop</finalName>          <!-- BookShop.jar, not bookshop-
1.0.jar -->
  <sourceDirectory>src/main/java</sourceDirectory>  <!-- rarely needed -->
  <resources>
    <resource><directory>src/main/resources</directory></resource>
  </resources>
  <testResources>…</testResources>         <!-- same, for test files -->
  <plugins>
    <plugin>
      <groupId>…</groupId><artifactId>…</artifactId><version>…</version>
      <configuration>…</configuration>     <!-- plugin settings -->
      <executions>
        <execution>
          <id>my-run</id>
          <phase>package</phase>           <!-- when it runs -->
          <goals><goal>shade</goal></goals> <!-- what runs -->
        </execution>
      </executions>
      <dependencies>…</dependencies>       <!-- deps for the plugin itself -->
    </plugin>
  </plugins>
  <pluginManagement>…</pluginManagement>   <!-- pin plugin versions, like
dependencyManagement -->
</build>
```

The `execution` block — id, phase, goals — is the single most useful pattern here: it binds any plugin goal to any lifecycle phase (chapter 8 used it for shade).

6 · Profiles

Named POM overrides, activated with `mvn package -Pdev` — chapter 9 in full.

7 · Repositories & pluginRepositories

Extra places to download from, beyond Maven Central — `<pluginRepositories>` is the same thing for plugins:

```
<repositories>
  <repository>
    <id>myrepo</id>
    <url>https://repo.mycompany.com/releases</url>
  </repository>
</repositories>
```

8 · Modules & parent

The two halves of a multi-module build (chapter 10): the parent lists `<modules>`, each child points back with `<parent>` and inherits its groupId, version, properties, and managed versions. Spring Boot projects use `<parent>` to inherit from `spring-boot-starter-parent`.

```
<modules>
  <module>core</module>
  <module>api</module>
</modules>

<parent>
  <groupId>com.example</groupId>
  <artifactId>bookshop</artifactId>
  <version>1.0-SNAPSHOT</version>
</parent>
```

9 · Publishing & project metadata

Tags you'll meet in open-source and released libraries — descriptive rather than build-changing, except the last:

TAG	PURPOSE
`<scm>`	Source-control (Git) URLs for the project
`<developers>`	Who maintains the project
`<licenses>`	Distribution license(s)
`<organization>`	Owning organization's name and URL
`<distributionManagement>`	Where `mvn deploy` uploads to (chapter 11)
`<reporting>`	Report plugins for `mvn site`